

Repository Analysis Report

Date: 2026-04-25

Scope and method

This report documents the codebase in `/Users/utilizador/cernbox/tau_analysis` as a full repository-level analysis note. It is organized per physics channel and by analysis stage. The goal is to explain:

- what each major code area does;
- which parts appear to be core analysis code versus legacy/prototype/support material;
- how the physics selection, proton treatment, weights, TMVA, and Combine stages fit together;
- what the current documentation and slide material say;
- where the code and the documentation agree, and where they diverge.

The report is based on:

- repository code under `MuTau_channel/`, `ETau_channel/`, `TauTau_Channel/`, `fase0_code/`, `fase1_code/`, `background/`, and `plots/`;
- current Markdown documentation and slides in `documentation/` and `MuTau_channel/slides/`;
- repository summaries in `README.md` and `project_status.md`.

Important limitation:

- Many PDFs exist in the repository, but in this environment they are not directly text-extractable with the available shell tools. Where a PDF has a Markdown counterpart, that Markdown was used as the authoritative machine-readable source. The MuTau slides and normalization notes are well covered by Markdown. Older PDF-only files are included here as part of the documentation inventory, but not quoted line-by-line.

Executive summary

This repository is a CMS central exclusive di-tau analysis workspace centered on 2018 UL data and PPS proton tagging. The dominant physics idea is to study an exclusive or semi-exclusive central system reconstructed from tau-containing final states and compare the central system to forward proton information measured in PPS.

The repository is not a single clean package. It is a working analysis area with:

- a relatively mature MuTau workflow, including skimming, event selection, proton handling, normalization studies, signal processing, TMVA, and Combine;
- an ETau branch that mixes older MuTau-like naming with newer `RDataFrame` and `awkward/uproot`-based implementations;
- a TauTau branch with more extensive `ROOT/C++` macros, control-region tooling, systematic variations, and MVA infrastructure;
- older `fase0_code/` and `fase1_code/` directories that look like predecessors or generic prototypes of the later channel-specific workflows;
- generated plots, datacards, `ROOT` outputs, weights, and environment material.

The most mature and best documented physics path is the MuTau channel. It is also the only channel for which the repository contains a coherent chain from event processing to limit setting, with active Markdown documentation explaining normalization, proton mixing, TMVA, and Combine.

The repository also shows a transition in analysis philosophy:

1. early phase: add pileup protons to MC and sometimes to derived data products after central selection;
2. later phase: save proton variables already in phase-0/phase-1 outputs and treat proton acceptance and normalization more explicitly;
3. current mature phase: build shapes, train multivariate discriminants, and run statistical inference with Combine.

Physics context

The common physics target across the repository is exclusive or proton-tagged di-tau production, especially gamma-gamma to tau-tau with one of the visible final states:

- MuTau: one muon plus one hadronic tau;
- ETau: one electron plus one hadronic tau;
- TauTau: two hadronic taus.

The forward proton logic is central:

- PPS multi-RP protons are preferred over single-RP protons;
- events are usually required to have proton activity on both arms;
- the proton fractional momentum losses ξ_1, ξ_2 are used to form matching observables against the central system:
 - mass matching: $m_{\text{central}} - 13000 \cdot \sqrt{\xi_1 \cdot \xi_2}$;
 - rapidity matching: $y_{\text{central}} - 0.5 \cdot \log(\xi_1 / \xi_2)$.

These are exactly the observables expected in exclusive production. For a clean exclusive topology, the centrally reconstructed state and the forward protons should be kinematically consistent. That is why the code and slides emphasize:

- central invariant mass;
- central rapidity;
- acoplanarity;
- proton-arm occupancy;
- PPS matching observables.

The selection thresholds seen repeatedly in the MuTau and ETau code are also physically sensible for a high-mass exclusive topology:

- hard tau threshold, typically $p_T(\text{tau}) > 100 \text{ GeV}$;
- hard lepton threshold, typically $p_T(\mu/e) > 35 \text{ GeV}$;
- opposite-sign requirement in signal-like regions;
- same-sign inversion for QCD control samples;
- $\Delta\eta > 0.4$;
- $|\eta| < 2.4$.

Code inventory

Authored analysis code outside the CMSSW tree is substantial. By file extension and top-level area:

- MuTau_channel: 24 Python, 23 C++, 17 ROOT macros, 5 shell;

- `ETau_channel`: 6 Python, 28 C++, 4 ROOT macros, 12 shell;
- `TauTau_Channel`: 4 Python, 44 C++, 10 ROOT macros, 11 shell;
- `background`: 1 Python, 5 C++, 2 shell;
- `fase0_code`: 1 Python, 7 C++, 3 shell;
- `fase1_code`: 2 Python, 4 C++, 4 shell;
- `plots`: 3 Python.

The repository also contains a full `fase1_code/CMSSW_12_2_4` area. That is mostly framework and environment material, not authored analysis logic, and should not be treated as channel analysis code.

Global repository structure

1. Documentation and status

- `README.md`: current checkpoint-style overview focused on MuTau plots, cuts, proton handling, event counts, and the proton-enrichment issue.
- `project_status.md`: broad project snapshot describing three channels, phase-0/phase-1/photon-tagging structure, and open issues.
- `documentation/`: presentation and note area, mostly mirrored by Markdown files for the MuTau workflow.
- `MuTau_channel/slides/`: the most useful current documentation set. These slides document normalization, signal processing, TMVA, and Combine in detail.

2. Channel analysis areas

- `MuTau_channel/`: most complete channel.
- `ETau_channel/`: partially modernized, but less integrated and less documented.
- `TauTau_Channel/`: extensive but more heterogeneous; a lot of ROOT/C++ macros, control-region work, and systematic variations.

3. Historical or generic stages

- `fase0_code/`: older or channel-agnostic skimming and proton studies.
- `fase1_code/`: older phase-1 selection studies and plots, plus the CMSSW area.
- `background/`: older QCD-oriented scripts and utilities.
- `plots/`: generic plotting and correlation scripts.

Common analysis architecture

Across channels, the workflow is conceptually similar:

1. Phase 0 / skimming
 - read NanoAOD or analyzer trees;
 - apply basic trigger/object presence cuts;
 - define leading-object kinematics and event variables;
 - keep a reduced tree.
2. Phase 1 / analysis selection
 - apply ID working points;
 - apply kinematic thresholds;

- enforce charge requirement;
 - save selected variables;
 - for MC, compute event weights and scale factors.
3. Proton treatment
 - either use real proton collections already saved in the selected tree;
 - or enrich MC with pileup-proton information from a proton pool;
 - then derive proton-arm and matching observables.
 4. Plotting and shape production
 - fill distributions for mass, rapidity, acoplanarity, matching variables, etc.;
 - compare data, DY, ttbar, QCD, and signal.
 5. MVA / TMVA
 - train discriminants using central and proton-matching observables;
 - apply trained models to all samples;
 - compare method performance.
 6. Statistical inference
 - build ROOT shapes and datacards;
 - run Combine for limits, significance, GoF, and nuisance studies.

This structure is explicit in the MuTau slides and strongly reflected in the code.

MuTau channel

Role in the repository

MuTau is the primary analysis channel. It has:

- the most complete skimming and selection pipeline;
- the clearest proton-enrichment workflow;
- dedicated normalization validation;
- signal production code;
- TMVA training and application;
- Combine datacards and outputs;
- multiple Markdown notes documenting the physics and technical choices.

Key code groups

Phase 0 skimming Representative files:

- MuTau_channel/fase0_data_mutau.py
- MuTau_channel/fase0_data.py
- MuTau_channel/fase0_dy.py
- MuTau_channel/fase0.py
- MuTau_channel/fase0_qcd_data.py

MuTau_channel/fase0_data_mutau.py is the clearest current reference for data skimming. It:

- builds robust XRootD access and resume logic;
- compiles a lumi mask from dadosluminosidade.txt;
- requires certified lumi and HLT_IsoMu24;
- defines leading muon and tau quantities;
- computes central-system mass, pT, rapidity, acoplanarity;

- stores jet variables, event weights, and the full proton collections;
- snapshots a reduced tree for later phases.

Code anchors:

- lumi filtering and trigger: `MuTau_channel/fase0_data_mutau.py:227`
- kinematic definitions: `MuTau_channel/fase0_data_mutau.py:239`
- proton collection preservation: `MuTau_channel/fase0_data_mutau.py:273`

Physics meaning:

- Phase 0 does not yet impose the full high-purity signal selection.
- Its main role is to reduce the event size while preserving enough information for later selections, especially PPS-related branches.

Phase 1 selection Representative files:

- `MuTau_channel/fase1_data.py`
- `MuTau_channel/fase1_dy.py`
- `MuTau_channel/fase1_qcd.py`
- `MuTau_channel/fase1_ttjets.py`
- `MuTau_channel/fase1_ttjets_new.py`

The core phase-1 cuts are consistent across signal-like samples:

- `mu_id >= 3;`
- `tau_id1 > 63;`
- `tau_id2 > 7;`
- `tau_id3 > 1;`
- `DeltaR > 0.4;`
- `|eta| < 2.4;`
- `mu_pt > 35 GeV;`
- `tau_pt > 100 GeV;`
- opposite-sign for signal-like regions;
- same-sign for QCD.

Code anchors:

- data phase-1 cuts: `MuTau_channel/fase1_data.py:122`
- DY phase-1 cuts: `MuTau_channel/fase1_dy.py:211`
- QCD same-sign inversion: `MuTau_channel/fase1_qcd.py:122`
- ttbar phase-1 cuts: `MuTau_channel/fase1_ttjets.py:203`

Physics meaning:

- The very tight tau ID plus hard tau pT is targeted at reducing fake-tau and low-mass background contamination.
- Same-sign MuTau is used as a QCD-enriched control sample.

Muon scale factors and event weights The repository documentation and code agree that MC event weights are built from:

- generator weight;

- muon trigger scale factor;
- muon ID/isolation scale factor;
- muon reconstruction scale factor.

In code:

- DY uses `event_weight = generator_weight * mu_trig_sf * mu_idiso_sf * mu_reco_sf` in `MuTau_channel/fase1_dy.py:226`.
- ttbar uses `event_weight = 0.15 * mu_trig_sf * mu_idiso_sf * mu_reco_sf` in `MuTau_channel/fase1_ttjets.py:217`.

This is one of the most important implementation asymmetries in the repository.

Interpretation:

- DY keeps the generator weight explicitly and is later scaled by $L \cdot \sigma / \text{Sum}_w$.
- ttbar instead bakes in an empirical or precomputed normalization factor 0.15 already during phase 1.

The documentation recognizes this as technically inconsistent and something that should be standardized, even if the final normalization can be made numerically reasonable.

Proton handling in MuTau MuTau contains two proton paradigms:

1. real proton variables stored directly in skims and selected trees;
2. proton-pool mixing applied to MC to emulate pileup PPS protons.

The direct-proton path is visible already in phase 0, where full proton collections are kept:

- `nproton_multi`, `nproton_single`;
- `proton_multi_xi`, `proton_multi_arm`, `proton_multi_t`;
- `proton_multi_thetaX`, `proton_multi_thetaY`;
- timing branches;
- `proton_single_xi`.

The proton-mixing path is implemented in `MuTau_channel/merge_pp_mutau.py`.

What it does:

- opens the proton pool and an input MC tree;
- copies central-event branches;
- creates `xi_arm1_1`, `xi_arm1_2`, `xi_arm2_1`, `xi_arm2_2`;
- always assigns at least one proton per arm;
- optionally assigns second protons according to fixed probabilities;
- multiplies the incoming `event_weight` by a fixed proton-acceptance factor.

Code anchors:

- proton pool configuration: `MuTau_channel/merge_pp_mutau.py:10`
- fixed proton acceptance factor: `MuTau_channel/merge_pp_mutau.py:15`
- output xi branches: `MuTau_channel/merge_pp_mutau.py:166`
- final weight logic: `MuTau_channel/merge_pp_mutau.py:236`

Physics meaning:

- the proton pool is meant to inject realistic random pileup proton activity into MC, since inclusive MC samples do not contain the PPS pileup environment seen in data;
- the two-arm requirement approximates the topology needed for PPS matching;
- the acceptance factor attempts to correct the fact that the mixing algorithm always injects a proton configuration, while in real data many events have no usable protons on one or both arms.

Important documented evolution:

- older notes mention a fixed 0.13 acceptance;
- newer MuTau documentation argues for 0.245 measured directly from data.

This discrepancy is central to the repository’s normalization history.

Plotting and shape production MuTau plotting is spread across several files:

- MuTau_channel/plot_m.cpp
- MuTau_channel/plot_m.py
- MuTau_channel/plot_all_variables.py
- MuTau_channel/plot_simple.py
- MuTau_channel/plot_proton_comparisons.py
- MuTau_channel/MuTauPlots*.C

The key shape-building macro for statistical inference is MuTau_channel/save_shapes.cpp.

Its role:

- opens data, DY, ttbar, QCD, and signal trees;
- fills eight principal histograms:
 - acoplanarity;
 - central invariant mass;
 - system pT;
 - system rapidity;
 - rapidity matching;
 - mass matching;
 - tau pT;
 - a branch labeled `met` but commented as effectively related to muon pT in one code comment;
- for data and QCD, requires real two-arm proton presence from stored proton vectors;
- for MC and signal, requires valid mixed `xi_arm1_1` and `xi_arm2_1`;
- scales DY by $1.004\text{e-}4$;
- leaves ttbar at scale 1.0 because its 0.15 was already included upstream.

Code anchors:

- normalization constants: MuTau_channel/save_shapes.cpp:10
- file inputs: MuTau_channel/save_shapes.cpp:52
- histogram choices and matching variables: MuTau_channel/save_shapes.cpp:75
- data proton requirement: MuTau_channel/save_shapes.cpp:144
- DY weighted fill: MuTau_channel/save_shapes.cpp:209
- ttbar weighted fill: MuTau_channel/save_shapes.cpp:231

Physics meaning:

- the mass and rapidity matching observables are the explicit bridge between central reconstruction and PPS proton kinematics;
- this is where the analysis stops being a generic lepton-tau selection and becomes an exclusive-production search.

Signal processing The most sophisticated single file in the repository is `MuTau_channel/sinal.py`.

This file is effectively a self-contained signal production and weighting chain. The corresponding slide note in `MuTau_channel/slides/slides_weights.md` explains it in detail.

Key functions of `sinal.py`:

- reads a signal ntuple with SM and BSM weights;
- computes normalization from luminosity and sigma;
- applies muon trigger/ID/reco scale factors;
- computes xi systematic up/down variations from PPS calibration graphs;
- requires at least one proton in each arm;
- applies xangle-dependent θ_x acceptance cuts;
- applies era-dependent pixel fiducial cuts;
- applies radiation-damage and multi-RP efficiency weights;
- stores central kinematics, proton kinematics, systematic branches, and a full `weights_bsm_sf[102]` array.

Code anchors:

- proton helper utilities: `MuTau_channel/sinal.py:156`
- xi systematics inputs: `MuTau_channel/sinal.py:188`
- branch definitions: `MuTau_channel/sinal.py:227`
- cutflow labels: `MuTau_channel/sinal.py:347`
- proton access from input ntuple: `MuTau_channel/sinal.py:414`
- proton-per-arm requirement: `MuTau_channel/sinal.py:496`
- xi systematic branch filling: `MuTau_channel/sinal.py:901`
- final stored kinematics: `MuTau_channel/sinal.py:941`

Physics meaning:

- this file models the signal in the most analysis-faithful way in the repository;
- it includes PPS fiducial and efficiency effects instead of treating protons only as a loose post-selection flag;
- it therefore contains the best implementation of the actual exclusive-physics logic.

TMVA and multivariate analysis MuTau has a complete TMVA pipeline:

- `MuTau_channel/TMVAClassification.C`
- `MuTau_channel/TMVAClassificationApplication.C`
- `MuTau_channel/TMVAClassification_fast.C`
- `MuTau_channel/TMVAClassification_scan.C`
- `MuTau_channel/mva_comparison.cpp`
- `MuTau_channel/run_bdt_all.C`
- `MuTau_channel/run_bdt_all.sh`

The active training file `MuTau_channel/TMVAClassification.C` enables:

- BDT;
- Likelihood;
- Fisher.

Code anchors:

- enabled methods: `MuTau_channel/TMVAClassification.C:82`, `MuTau_channel/TMVAClassification.C:183`
- training samples: `MuTau_channel/TMVAClassification.C:255`

The application file uses nine variables:

- `sist_rap`;
- `acop`;
- `sist_pt`;
- `mu_pt`;
- `tau_pt`;
- `sist_mass`;
- `sist_mass - sqrt(s)*sqrt(xi1*xi2)` implemented as `sist_mass - sqrt(13000^2*xi1*xi2)`;
- `met_pt`;
- `sist_rap - 0.5*log(xi1/xi2)`.

Code anchors:

- TMVA reader variables: `MuTau_channel/TMVAClassificationApplication.C:127`
- proton-validated input use: `MuTau_channel/TMVAClassificationApplication.C:314`

Physics meaning:

- the MVA combines standard topology variables with proton-matching variables;
- this is exactly the right direction for an exclusive analysis because it lets the discriminator exploit both central and forward information.

Combine and statistical inference MuTau contains the only fully visible Combine workflow in the repository.

Key files:

- `MuTau_channel/save_shapes.cpp`
- `MuTau_channel/datacards/mutau_mass_shape.txt`
- `MuTau_channel/datacards/mutau_bdt_shape.txt`
- `MuTau_channel/datacards/mutau_likelihood_shape.txt`
- `MuTau_channel/datacards/plot_single_limit.py`

The documentation in `documentation/slides_combine.md` and `MuTau_channel/slides/slides_tmva_combine` explains:

- shape-based datacards from ROOT histograms;
- nuisance parameters for luminosity, tau ID, muon systematics, proton xi, and background normalizations;
- `rateParam` usage for DY, ttbar, QCD;
- asymptotic limits and GoF/significance studies.

Observed results documented in the slides:

- mass-based limit median expected $r \sim 4000$;
- BDT-based median expected also reported around 4000;
- Likelihood observed result improved over BDT in the slide note, though the comparison table is still incomplete.

Physics interpretation:

- the analysis is currently far from SM sensitivity;
- the slides explicitly state that the search is excluding cross sections thousands of times the SM value;
- that is consistent with a low-yield exclusive final state plus substantial background/systematic limitations.

MuTau documentation synthesis

The MuTau Markdown notes are strong and mostly consistent:

- `documentation/NORMALIZATION_DOCUMENTATION.md` explains the normalization formula and generator-weight sums.
- `documentation/presentation_meeting.md` explains the historical bug where proton mixing overwrote event weights.
- `documentation/presentation_mutau_weights.md` explains the proton-pool and phase-0/phase-1 logic.
- `documentation/slides_combine.md` explains the Combine pipeline and mass-shape datacard.
- `documentation/slides_mutau.md` explains weight evolution and single-variable plots.
- `MuTau_channel/slides/slides_tmva_combine.md` explains the TMVA-to-Combine extension.
- `MuTau_channel/slides/slides_weights.md` explains signal modeling in `sinal.py`.

Main MuTau strengths:

- most complete end-to-end path;
- most explicit physics documentation;
- best connection between code and analysis intent.

Main MuTau weaknesses:

- normalization conventions are not fully unified across MC samples;
- proton acceptance value is historically inconsistent across notes;
- phase-1 data/QCD scripts have commented-out lumi/trigger filters and rely on phase-0 preservation;
- there is still overlap between old and new plotting conventions.

ETau channel

Role in the repository

ETau is present as a parallel channel but is clearly less mature and less unified than MuTau. It looks like a mixture of:

- older ROOT/C++ skims and macros;
- newer RDataFrame implementations;
- an awkward/uproot-based prototype for phase-1 data selection;
- analogous signal and TMVA scaffolding.

Key code groups

Representative files:

- `ETau_channel/etau_skim_rdf.cpp`
- `ETau_channel/fase0_data_rdf.py`
- `ETau_channel/fase1_data_rdf.py`
- `ETau_channel/nanotry*.cpp`
- `ETau_channel/TMVAClassification.C`
- `ETau_channel/TMVAClassificationApplication.C`
- `ETau_channel/sinal*.cpp`

Physics selection

The C++ RDataFrame skimmer `ETau_channel/etau_skim_rdf.cpp` is the cleanest ETau implementation.

It:

- requires at least one electron and one tau;
- requires leading electron ID;
- requires tau anti-jet, anti-electron, anti-muon working points;
- imposes $|\eta_{\text{ta}}| < 2.4$, $e_{\text{pt}} > 35$, $\tau_{\text{pt}} > 100$, opposite sign, and $\Delta R > 0.4$;
- computes system mass, p_T , rapidity, and acoplanarity;
- stores scalarized leading-object quantities.

Code anchors:

- selection chain: `ETau_channel/etau_skim_rdf.cpp:50`
- scalar conversion and kinematics: `ETau_channel/etau_skim_rdf.cpp:79`

The Python phase-0 style file `ETau_channel/fase0_data_rdf.py` reproduces a similar object definition and kinematics logic, although it appears to be mislabeled or repurposed because the parameter block points to `QCD_2018_UL.txt` while the output path names refer to ETau and even include a `mutau` fragment.

Code anchors:

- trigger and object filters: `ETau_channel/fase0_data_rdf.py:82`
- kinematic variable definitions: `ETau_channel/fase0_data_rdf.py:92`

The awkward/uproot phase-1 prototype `ETau_channel/fase1_data_rdf.py` differs from MuTau in one important way:

- it applies a simple PPS requirement of at least two valid protons rather than an explicit one-per-arm criterion.

Code anchors:

- proton validity and ≥ 2 requirement: `ETau_channel/fase1_data_rdf.py:61`

Physics interpretation:

- ETau retains the same high-pT exclusive-style central selection as MuTau;
- however, its proton logic is currently less physically specific than the MuTau one-per-arm matching strategy.

ETau maturity assessment

Strengths:

- clear leading-object selection logic;
- modern RDataFrame implementation exists;
- a compact awkward/uproot phase-1 test exists.

Weaknesses:

- inconsistent naming and some evidence of copy/adaptation from MuTau;
- no equally mature normalization and Combine documentation;
- less explicit treatment of proton matching observables;
- no clearly documented end-to-end statistical workflow in the repository.

Conclusion:

ETau is a secondary analysis branch, usable as a structural parallel to MuTau but not yet documented or integrated at the same level.

TauTau channel

Role in the repository

TauTau is large and technically rich, but it is more heterogeneous than MuTau. It appears to host:

- phase-0 and phase-1 processing for hadronic tau pairs;
- tau-ID and tau-energy systematic handling;
- proton mixing;
- TMVA;
- multiple plotting modes;
- control-region studies and combined control-region figures.

It is likely the most ROOT/C++-centric part of the repository.

Key code groups

Representative files:

- `TauTau_Channel/nanotry_fase1_Dados.cpp`
- `TauTau_Channel/nanotry_fase1_DY.cpp`
- `TauTau_Channel/nanotry_fase1_QCD.cpp`
- `TauTau_Channel/nanotry_fase1_ttjets.cpp`
- `TauTau_Channel/mergePileupProtons_Joao_2018_bkg_tau_tau.cpp`
- `TauTau_Channel/TMVAClassification.C`
- `TauTau_Channel/plot_m.cpp`

- `TauTau_Channel/combine_cr_plots.py`
- `TauTau_Channel/make_tree_syst_prot_xi*/*.cpp`

Phase-1 selection

In data, `TauTau_Channel/nanotry_fase1_Dados.cpp` applies a straightforward hadronic tau-tau selection:

- both taus pass tight ID-like thresholds;
- $\Delta R > 0.4$;
- $|\eta| < 2.4$;
- variables are written to a reduced tree with `weight = 1.0`.

Code anchors:

- data weight definition: `TauTau_Channel/nanotry_fase1_Dados.cpp:26`
- tau-tau selection: `TauTau_Channel/nanotry_fase1_Dados.cpp:136`

In DY, `TauTau_Channel/nanotry_fase1_DY.cpp` is more elaborate:

- sets a nominal `weight = 1.81`;
- loads tau ID and tau-energy correction files;
- computes tau ID and tau-energy systematic variations;
- stores up/down-shifted tau pT, system mass, and system pT branches.

Code anchors:

- DY normalization seed: `TauTau_Channel/nanotry_fase1_DY.cpp:27`
- tau SF inputs: `TauTau_Channel/nanotry_fase1_DY.cpp:125`
- selected-event branch filling after cuts: `TauTau_Channel/nanotry_fase1_DY.cpp:187`

Physics meaning:

- TauTau emphasizes tau-specific systematics more strongly than MuTau;
- the hadronic final state needs tighter control of tau ID and energy scale, so this is expected.

Proton mixing

`TauTau_Channel/mergePileupProtons_Joao_2018_bkg_tau_tau.cpp` is the TauTau analog of MuTau proton mixing.

It:

- opens the 2018 proton pool;
- reads a tau-tau tree;
- assigns at least one proton per arm;
- optionally assigns second protons with the same probability pattern;
- writes `xi_arm1_1`, `xi_arm1_2`, `xi_arm2_1`, `xi_arm2_2`;
- writes a fixed `weight_corrector = 1.0 * 0.13`.

Code anchors:

- proton-pool input and fixed weight: `TauTau_Channel/mergePileupProtons_Joao_2018_bkg_tau_tau.cpp:`
- branch writing: `TauTau_Channel/mergePileupProtons_Joao_2018_bkg_tau_tau.cpp:82`
- proton assignment loops: `TauTau_Channel/mergePileupProtons_Joao_2018_bkg_tau_tau.cpp:121`

This file also contains at least one clear coding problem:

- there is an incomplete declaration `double pu_xi` without a semicolon near line 42.

That strongly suggests this particular file may not currently compile as-is, or it is an in-repo draft rather than a verified production binary source.

TMVA in TauTau

`TauTau_Channel/TMVAClassification.C` enables a much broader menu than `MuTau`:

- Cuts;
- Likelihood;
- PDERS;
- Fisher;
- MLP;
- BDT.

Code anchors:

- enabled methods: `TauTau_Channel/TMVAClassification.C:48`, `TauTau_Channel/TMVAClassification.C:76`, `TauTau_Channel/TMVAClassification.C:94`, `TauTau_Channel/TMVAClassification.C:122`, `TauTau_Channel/TMVAClassification.C:163`

Physics interpretation:

- `TauTau` seems to have been used as a broad MVA laboratory, perhaps because the channel has more challenging backgrounds and tau-specific systematics.

Control regions and plotting

`TauTau` includes explicit control-region tooling:

- `TauTau_Channel/combine_cr_plots.py`
- `TauTau_Channel/makeCRplots.C`
- `TauTau_Channel/plot_cr.cpp`
- multiple combined-control-region PDFs.

`TauTau_Channel/combine_cr_plots.py` explicitly describes:

- DY control region;
- TT control region;
- QCD control region;
- distributions of acoplanarity, mass, rapidity, and p_T .

Code anchors:

- CR descriptions in caption text: `TauTau_Channel/combine_cr_plots.py:129`

This is one of the strongest signs that `TauTau` has a serious background-modeling program, even if the repository does not contain the same polished end-to-end documentation that `MuTau` has.

TauTau maturity assessment

Strengths:

- rich ROOT/C++ implementation;
- explicit tau systematic branches;
- dedicated control-region tooling;
- broad TMVA experimentation.

Weaknesses:

- code quality is less uniform than in MuTau;
- some files appear draft-like or not fully maintained;
- normalization conventions are less clearly documented;
- the analysis flow is harder to reconstruct from code alone.

Conclusion:

TauTau is a substantial analysis branch, likely important scientifically, but it is not as repository-coherent as MuTau.

Legacy and support directories

fase0_code/

This directory contains early phase-0 studies:

- `fase0.cpp`, `ttJets_fase0.cpp`, `cep_fase0_2.cpp`;
- test and proton-check utilities;
- an older `fase0.py`.

These files likely predate the channel-specific reorganizations and are best interpreted as prototyping or early skimming studies.

fase1_code/

This directory contains:

- generic phase-1 C++ studies such as `dados_fase1.cpp`, `dy_fase1.cpp`, `mc_fase1_try.cpp`, `nanotry_fase1_QCD.cpp`;
- a plotting script `plots.py`;
- a full CMSSW_12_2_4 environment.

This looks like an older central workspace from which some channel-specific code evolved. It is useful historically, but it is not the best entry point for the current analysis.

background/

This area contains QCD-focused or generic background material:

- `QCD_fase0.cpp`;
- `nanotry_fase1_QCD.cpp`;
- branch-printing and test-parallel utilities;
- a simple plotting script.

It appears to be an older support area rather than an actively maintained standalone pipeline.

`plots/`

This area contains generic postprocessing tools:

- `corrs_matrix.py`;
- `comparisons_new.py`;
- `plots2.py`.

These are analysis-support scripts for correlations and comparisons rather than event-processing code.

Documentation and slide synthesis

Most useful current docs

The best current documentation is:

- `project_status.md`
- `documentation/NORMALIZATION_DOCUMENTATION.md`
- `documentation/presentation_meeting.md`
- `documentation/presentation_mutau_weights.md`
- `documentation/slides_combine.md`
- `documentation/slides_mutau.md`
- `MuTau_channel/slides/slides_tmva_combine.md`
- `MuTau_channel/slides/slides_weights.md`

Together they describe:

- the phase-0/phase-1 architecture;
- the proton-pool idea;
- the weight bug and its fix;
- the MC normalization constants;
- the signal weighting chain;
- the TMVA feature set;
- the Combine datacards and results.

Main repository-wide analysis story from the docs

1. The collaboration wants a proton-tagged di-tau search in 2018 UL data.
2. MuTau is the lead implementation and serves as the template.
3. MC samples need help to emulate PPS protons, so a proton-pool enrichment is used.
4. That enrichment historically caused a normalization bug because it overwrote event weights.
5. The analysis is moving toward a cleaner treatment:
 - preserve proton variables in reduced trees;
 - measure proton acceptance from data;
 - normalize MC with explicit $L \cdot \sigma / \text{Sum}_w$;
 - use proton matching variables in TMVA and Combine.
6. Current sensitivity is still far from the SM signal.

Main tensions between docs and code

1. Proton acceptance: 0.13 vs 0.245 The documentation evolves from 0.13 to 0.245, but `merge_pp_mutable.py` still documents and applies a fixed acceptance factor through a single constant path. The notes clearly argue that 0.245 is the more recent data-driven value, but the historical 0.13 convention still exists in slides and comments.

2. ttbar normalization remains special-cased The docs explicitly call out that ttbar already includes a 0.15 factor in phase 1. The code confirms this. That means MuTau MC normalization is not implemented in a fully symmetric way across DY and ttbar.

3. Data/QCD phase-1 trigger and lumi filtering are commented out The code shows that the phase-1 data and QCD scripts rely on phase-0 preservation of those selections. That is probably acceptable operationally, but it means the phase-1 scripts are not independently self-documenting as complete selections.

4. Repository status docs are more polished than some channel branches `project_status.md` presents the repository as a clean three-channel project. In reality, MuTau is much more mature than ETau, and ETau is cleaner than some parts of TauTau, while the older `fase0_code/`, `fase1_code/`, and `background/` directories remain mixed into the same workspace.

Scientific interpretation

From the code and notes together, the intended physical signal is:

- a central tau-containing system consistent with exclusive gamma-gamma production;
- accompanied by forward proton tags measured in PPS;
- with central-forward kinematic consistency tested via `m_X` and rapidity matching.

The logic of the observables is strong:

- `acop`: exclusive processes tend to be more back-to-back;
- `sist_pt`: exclusive systems should prefer lower additional recoil;
- `sist_mass - 13000*sqrt(xi1*xi2)`: checks proton-central consistency in mass;
- `sist_rap - 0.5*log(xi1/xi2)`: checks proton-central consistency in rapidity;
- proton-arm occupancy: rejects nonexclusive-like topologies.

The repository also reflects the main experimental difficulty:

- real PPS proton activity is strongly contaminated or dominated by pileup;
- inclusive MC lacks that realistic proton environment;
- therefore proton acceptance and proton-pool mixing dominate a large part of the systematic and normalization discussion.

That is why so much code and documentation is devoted not just to lepton/tau selection, but to:

- proton enrichment;
- proton acceptance measurement;
- proton xi systematics;
- PPS fiducial and efficiency modeling.

Code quality and maintainability assessment

Strong parts

- MuTau current-phase Python + ROOT workflow is understandable and documented.
- `signal.py` contains a serious signal modeling implementation with physically meaningful PPS treatment.
- `save_shapes.cpp` and the Combine datacards make the statistical workflow explicit.
- The documentation is unusually good for a personal or working HEP repository.

Weak parts

- The repository is not cleanly separated into production, prototype, and generated artifacts.
- Normalization conventions are not fully standardized across all samples and channels.
- Some files are clearly experimental or stale.
- Some channel code still contains copy/paste residue or inconsistent naming.
- At least some TauTau files appear not to be compile-clean as committed.

Practical recommendation for future organization

The repository would benefit from a split into:

1. `analysis/mutau/`
2. `analysis/etau/`
3. `analysis/tautau/`
4. `legacy/`
5. `docs/`
6. `outputs/`
7. `env/` or external CMSSW setup

This would make it much easier to distinguish:

- active analysis code;
- historical studies;
- generated products;
- framework dependencies.

Final channel-by-channel assessment

MuTau

Status:

- primary analysis branch;
- most complete and best documented;
- only channel with a clearly visible full path from event processing to limits.

Best entry points:

- `MuTau_channel/fase0_data_mutau.py`
- `MuTau_channel/fase1_dy.py`
- `MuTau_channel/fase1_ttjets.py`
- `MuTau_channel/merge_pp_mutau.py`

- `MuTau_channel/sinal.py`
- `MuTau_channel/save_shapes.cpp`
- `MuTau_channel/TMVAClassification.C`
- `MuTau_channel/TMVAClassificationApplication.C`

Physics readiness:

- highest in repository.

ETau

Status:

- secondary branch;
- structurally similar to MuTau but less integrated;
- modernized pieces exist but are not yet assembled into an equally complete analysis chain.

Best entry points:

- `ETau_channel/etau_skim_rdf.cpp`
- `ETau_channel/fase0_data_rdf.py`
- `ETau_channel/fase1_data_rdf.py`

Physics readiness:

- moderate, but behind MuTau.

TauTau

Status:

- large, technically rich, and probably scientifically important;
- less uniform in code quality and less coherently documented;
- strong on control regions and tau-specific systematics.

Best entry points:

- `TauTau_Channel/nanotry_fase1_Dados.cpp`
- `TauTau_Channel/nanotry_fase1_DY.cpp`
- `TauTau_Channel/mergePileupProtons_Joao_2018_bkg_tau_tau.cpp`
- `TauTau_Channel/TMVAClassification.C`
- `TauTau_Channel/combine_cr_plots.py`

Physics readiness:

- substantial but harder to validate quickly than MuTau.

Conclusion

This repository is a real analysis workspace for proton-tagged di-tau studies in CMS 2018 UL data, with MuTau as the flagship channel. The codebase captures the full chain from skimming to statistical inference, but mostly in the MuTau channel. ETau and TauTau are meaningful parallel branches, though they are less unified and less documented.

Scientifically, the repository is centered on exclusive-physics consistency between the central di-tau system and the PPS proton system. The most important observables are therefore not only lepton and tau kinematics, but also proton-arm occupancy, xi values, and proton-central matching variables in mass and rapidity.

The main technical story of the repository is the evolution from simple proton enrichment and empirical normalizations toward a more principled treatment of:

- saved proton variables;
- generator-weight normalization;
- data-driven proton acceptance;
- PPS systematic modeling;
- multivariate separation;
- Combine-based limit setting.

If one had to choose a single authoritative path through the repository, it would be:

1. MuTau phase-0 skimming;
2. MuTau phase-1 MC/data selections;
3. MuTau proton mixing and acceptance studies;
4. `signal.py` for signal modeling;
5. `save_shapes.cpp` for statistical inputs;
6. TMVA training/application;
7. Combine datacards and plots.

That path best represents both the physics goals and the current state of the code.